```python
def SIGNModel(input_shape):
    """
    Implementation of the SIGN.

    Arguments:
    input_shape -- shape of the images of the dataset

    Returns:
    model -- a Model() instance in Keras
    """
    ### START CODE HERE ###
    # Feel free to use the suggested outline in the text above to get started, and run through the whole
    # exercise (including the later portions of this notebook) once. The come back also try out other
    # network architectures as well.
    X_input = Input(shape=input_shape)
    X = ZeroPadding2D(padding=(1, 1))(X_input)
    X = Conv2D(32, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    X = ZeroPadding2D(padding=(1, 1))(X)
    X = Conv2D(16, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

#     X = ZeroPadding2D(padding=(1, 1))(X)
#     X = Conv2D(8, kernel_size=(2,2), strides=(1,1))(X)
#     X = BatchNormalization(axis=3)(X)
#     X = Activation('relu')(X)
#     X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    # FC
    X  = Flatten()(X)
    Y1 = Dense(6,activation=None)(X)
    Y  = Activation("softmax")(Y1)

    model = Model(inputs = X_input, outputs = Y, name='SIGNModel')
    ### END CODE HERE ###

    return model
```

模型架構

輸入 1080 張照片，每張為(64*64*3)維度

經過第一層 conv2D，padding=(1,1)、strides=(1,1)因此輸出長寬會與原本相同輸出維度(64,64,32)，其中 32 為我們設定的輸出量

batch normalization 將所得到的特徵優化，axis=3 表特徵維度(N=32 這層)

經過第一層 Maxpool，padding=”valid”表不加入 padding

(若使用 padding=”same”會將不夠的地方補 0)，strides=(2,2)因此輸出長寬會縮成一半，輸出=(32,32,32)。

以此類推第二層

第三層

flatten 將輸入為度(16,16,16)壓平，輸出為度=16*16*16=4096

特別需要注意的是 Dense 輸出，因為我們的 one-hot encoder 有 6 種型態，所以 Dense 輸出要放 6 而不是 1。

註：padding 使用”same”、”valid”

https://blog.csdn.net/wuzqChom/article/details/74785643

將整個模型結構打映出來，可以使用 modelname.summary()映出

```
SIGNModel.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_14 (InputLayer)        (None, 64, 64, 3)         0
_________________________________________________________________
zero_padding2d_30 (ZeroPaddi (None, 66, 66, 3)         0
_________________________________________________________________
conv2d_30 (Conv2D)           (None, 64, 64, 32)        896
_________________________________________________________________
batch_normalization_30 (Batc (None, 64, 64, 32)        128
_________________________________________________________________
activation_43 (Activation)   (None, 64, 64, 32)        0
_________________________________________________________________
max_pooling2d_30 (MaxPooling (None, 32, 32, 32)        0
_________________________________________________________________
zero_padding2d_31 (ZeroPaddi (None, 34, 34, 32)        0
_________________________________________________________________
conv2d_31 (Conv2D)           (None, 32, 32, 16)        4624
_________________________________________________________________
batch_normalization_31 (Batc (None, 32, 32, 16)        64
_________________________________________________________________
activation_44 (Activation)   (None, 32, 32, 16)        0
_________________________________________________________________
max_pooling2d_31 (MaxPooling (None, 16, 16, 16)        0
_________________________________________________________________
flatten_14 (Flatten)         (None, 4096)              0
_________________________________________________________________
dense_11 (Dense)             (None, 6)                 24582
_________________________________________________________________
activation_45 (Activation)   (None, 6)                 0
=================================================================
Total params: 30,294
Trainable params: 30,198
Non-trainable params: 96
```

討論 parameter 計算

conv2D 公式為：(輸入通道數*卷積長*卷積寬+1)*輸出數

連階層公式為：(輸入維度+1)*輸出維度

batch normalization 公式為：特徵維度(N)*2*2，其中 2N 不可訓練，2N 可訓練

第一層 conv2D

輸入維度(64,64,3)其通道數=3，卷積大小(3,3)，輸出 32 個

因此參數量=(3*3*3+1)*32=896

第一層 batch normalization

特徵維度 N=32，32*2*2=128，64 個可訓練，64 個不可訓練

第二層 conv2D

輸入維度(32,32,32)其通道數=32，卷積大小(3,3)，輸出 32 個

因此參數量=(32*3*3+1)*16=4624

第二層 batch normalization

特徵維度 N=16，16*2*2=64，32 個可訓練，32 個不可訓練

全連接層 Dense

輸入通道數=4096，輸出 6 個，(4096+1)*6=24582

經由以上計算可得全部參數量為 896+128+4624+64+24582 = *30,294*

而其中經過 batch normalization 有各一半的參數能否訓練=64+32=96，

註：參數量計算

conv2D、Dense

https://blog.csdn.net/ybdesire/article/details/85217688

batch normalization

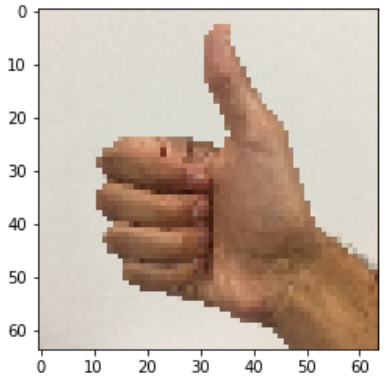
https://blog.csdn.net/C_chuxin/article/details/86592364

訓練結果

```
Epoch 1/10
1080/1080 [==============================] - 40s 37ms/step - loss: 2.8424 - acc: 0.4139
Epoch 2/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.7213 - acc: 0.7491
Epoch 3/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.4330 - acc: 0.8454
Epoch 4/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.3035 - acc: 0.9120
Epoch 5/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.2114 - acc: 0.9380
Epoch 6/10
1080/1080 [==============================] - 38s 36ms/step - loss: 0.1430 - acc: 0.9704
Epoch 7/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.1080 - acc: 0.9815
Epoch 8/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.0888 - acc: 0.9852
Epoch 9/10
1080/1080 [==============================] - 38s 35ms/step - loss: 0.0626 - acc: 0.9963
Epoch 10/10
1080/1080 [==============================] - 39s 36ms/step - loss: 0.0542 - acc: 0.9954
```

```
120/120 [==============================] - 1s 9ms/step
SIGNModel
Loss = 0.18234131236871085
Test Accuracy = 0.949999996026357
```

The figure is 3



可以見到第一次嘗試，訓練資料就有 99.54%的正確率，

測試資料也有 95%正確率，已經算是很成功的模型，

試著調整其他參數，看對於模型會有甚麼影響。

調整第二層 conv2D 從 16->32，這個舉動我認為在少層且資料量少的網路中效益並不大，反而拖慢訓練速度，在第二層抓取更多特徵理論上正確率應該要提高，因為電腦有更多參考點可以判讀，但如前面所述，資料量過少且太少層，效果應不顯著，實作驗證會不會打臉自己。

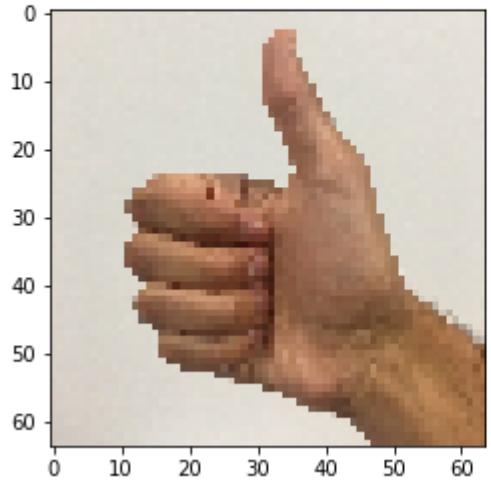
```
    X_input = Input((64, 64, 3))
    X = ZeroPadding2D(padding=(1, 1))(X_input)
    X = Conv2D(32, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    X = ZeroPadding2D(padding=(1, 1))(X)
    X = Conv2D(32, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

#     X = ZeroPadding2D(padding=(1, 1))(X)
#     X = Conv2D(8, kernel_size=(2,2), strides=(1,1))(X)
#     X = BatchNormalization(axis=3)(X)
#     X = Activation('relu')(X)
#     X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    # FC
    X  = Flatten()(X)
    Y1 = Dense(6,activation=None)(X)
    Y  = Activation("softmax")(Y1)

    model = Model(inputs = X_input, outputs = Y, name='SIGNModel')
    model.compile(optimizer = "sgd", loss = "categorical_crossentropy", metrics = ["accuracy"])
    model.fit(X_train,Y_train, epochs = 10, batch_size=16 )
```

模型架構

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_16 (InputLayer)        (None, 64, 64, 3)         0
_________________________________________________________________
zero_padding2d_34 (ZeroPaddi (None, 66, 66, 3)         0
_________________________________________________________________
conv2d_34 (Conv2D)           (None, 64, 64, 32)        896
_________________________________________________________________
batch_normalization_34 (Batc (None, 64, 64, 32)        128
_________________________________________________________________
activation_49 (Activation)   (None, 64, 64, 32)        0
_________________________________________________________________
max_pooling2d_34 (MaxPooling (None, 32, 32, 32)        0
_________________________________________________________________
zero_padding2d_35 (ZeroPaddi (None, 34, 34, 32)        0
_________________________________________________________________
conv2d_35 (Conv2D)           (None, 32, 32, 32)        9248
_________________________________________________________________
batch_normalization_35 (Batc (None, 32, 32, 32)        128
_________________________________________________________________
activation_50 (Activation)   (None, 32, 32, 32)        0
_________________________________________________________________
max_pooling2d_35 (MaxPooling (None, 16, 16, 32)        0
_________________________________________________________________
flatten_16 (Flatten)         (None, 8192)              0
_________________________________________________________________
dense_13 (Dense)             (None, 6)                 49158
_________________________________________________________________
activation_51 (Activation)   (None, 6)                 0
=================================================================
Total params: 59,558
Trainable params: 59,430
Non-trainable params: 128
_________________________________________________________________
```

訓練結果

```
Epoch 1/10
1080/1080 [==============================] - 65s 60ms/step - loss: 8.8812 - acc: 0.2981
Epoch 2/10
1080/1080 [==============================] - 63s 58ms/step - loss: 3.8333 - acc: 0.5963
Epoch 3/10
1080/1080 [==============================] - 63s 59ms/step - loss: 2.8709 - acc: 0.7593
Epoch 4/10
1080/1080 [==============================] - 64s 59ms/step - loss: 2.7941 - acc: 0.8037
Epoch 5/10
1080/1080 [==============================] - 66s 61ms/step - loss: 2.7170 - acc: 0.7889
Epoch 6/10
1080/1080 [==============================] - 61s 57ms/step - loss: 0.6232 - acc: 0.8481
Epoch 7/10
1080/1080 [==============================] - 61s 57ms/step - loss: 0.1730 - acc: 0.9454
Epoch 8/10
1080/1080 [==============================] - 58s 54ms/step - loss: 0.1370 - acc: 0.9574
Epoch 9/10
1080/1080 [==============================] - 52s 48ms/step - loss: 0.0622 - acc: 0.9796
Epoch 10/10
1080/1080 [==============================] - 42s 39ms/step - loss: 0.0363 - acc: 0.9935

<keras.callbacks.History at 0x1ee9746a5c0>

120/120 [==============================] - 1s 10ms/step
model
Loss = 0.49883023301760354
Test Accuracy = 0.8333333293596904
The figure is 2
```



可以看到訓練資料依然有 99%的正確率，但測試資料剩下 83%正確率，出現 overfitting，而這也證明了相同 epoch 增加特徵抓取，不一定能提高正確率，很明顯地看到 Loss 比前次還要高很多，因此若提高 epoch 理應還能提高正確率。

若是調整 mini batch size 從 16->64

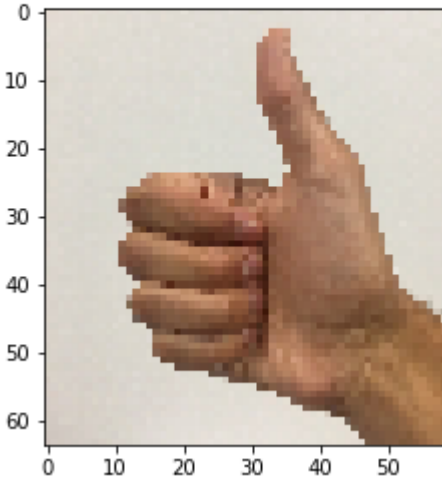
```
    X_input = Input((64, 64, 3))
    X = ZeroPadding2D(padding=(1, 1))(X_input)
    X = Conv2D(32, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    X = ZeroPadding2D(padding=(1, 1))(X)
    X = Conv2D(16, kernel_size=(3,3), strides=(1,1))(X)
    X = BatchNormalization(axis=3)(X)
    X = Activation('relu')(X)
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

#     X = ZeroPadding2D(padding=(1, 1))(X)
#     X = Conv2D(8, kernel_size=(2,2), strides=(1,1))(X)
#     X = BatchNormalization(axis=3)(X)
#     X = Activation('relu')(X)
#     X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

    # FC
    X  = Flatten()(X)
    Y1 = Dense(6,activation=None)(X)
    Y  = Activation("softmax")(Y1)

    model = Model(inputs = X_input, outputs = Y, name='SIGNModel')
    model.compile(optimizer = "sgd", loss = "categorical_crossentropy", metrics = ["accuracy"])
    model.fit(X_train,Y_train, epochs = 10, batch_size=64 )
```

```
Epoch 1/10
1080/1080 [==============================] - 44s 41ms/step - loss: 3.6939 - acc: 0.2574
Epoch 2/10
1080/1080 [==============================] - 44s 41ms/step - loss: 1.4154 - acc: 0.5037
Epoch 3/10
1080/1080 [==============================] - 47s 44ms/step - loss: 1.0456 - acc: 0.6537
Epoch 4/10
1080/1080 [==============================] - 44s 41ms/step - loss: 0.8799 - acc: 0.6722
Epoch 5/10
1080/1080 [==============================] - 44s 40ms/step - loss: 0.6285 - acc: 0.7898
Epoch 6/10
1080/1080 [==============================] - 45s 41ms/step - loss: 0.5085 - acc: 0.8306
Epoch 7/10
1080/1080 [==============================] - 45s 41ms/step - loss: 0.3990 - acc: 0.8870
Epoch 8/10
1080/1080 [==============================] - 44s 41ms/step - loss: 0.4092 - acc: 0.8731
Epoch 9/10
1080/1080 [==============================] - 45s 41ms/step - loss: 0.2740 - acc: 0.9389
Epoch 10/10
1080/1080 [==============================] - 45s 41ms/step - loss: 0.2349 - acc: 0.9546
```

可以看到因為 size 加大到 64，等同圖片大小，視為不使用 mini batch，loss 下降緩慢，可能需更多 epoch 才能增加正確率。

```
Epoch 1/10
1080/1080 [==============================] - 42s 39ms/step - loss: 4.6281 - acc: 0.2259
Epoch 2/10
1080/1080 [==============================] - 43s 40ms/step - loss: 1.2585 - acc: 0.5352
Epoch 3/10
1080/1080 [==============================] - 43s 40ms/step - loss: 0.8665 - acc: 0.6981
Epoch 4/10
1080/1080 [==============================] - 44s 40ms/step - loss: 0.6839 - acc: 0.7806
Epoch 5/10
1080/1080 [==============================] - 50s 46ms/step - loss: 0.5945 - acc: 0.8019
Epoch 6/10
1080/1080 [==============================] - 45s 41ms/step - loss: 0.4319 - acc: 0.8806
Epoch 7/10
1080/1080 [==============================] - 43s 40ms/step - loss: 0.3829 - acc: 0.9009
Epoch 8/10
1080/1080 [==============================] - 46s 43ms/step - loss: 0.3252 - acc: 0.9130
Epoch 9/10
1080/1080 [==============================] - 44s 41ms/step - loss: 0.2753 - acc: 0.9435
Epoch 10/10
1080/1080 [==============================] - 43s 40ms/step - loss: 0.2757 - acc: 0.9306
120/120 [==============================] - 1s 12ms/step
```

```
Loss = 0.4841639757156372
Test Accuracy = 0.85
The figure is 3
```



把 size 縮小為 32，正確率稍微下降，可以看到 size 在 16、32 和 64 與正確率並無絕對關係。

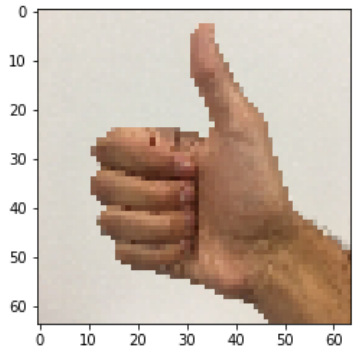
增加一層 conv2D 檢驗是否能提高正確率，將原本的模型註解的部分運作

```
X_input = Input((64, 64, 3))
X = ZeroPadding2D(padding=(1, 1))(X_input)
X = Conv2D(32, kernel_size=(3,3), strides=(1,1))(X)
X = BatchNormalization(axis=3)(X)
X = Activation('relu')(X)
X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

X = ZeroPadding2D(padding=(1, 1))(X)
X = Conv2D(16, kernel_size=(3,3), strides=(1,1))(X)
X = BatchNormalization(axis=3)(X)
X = Activation('relu')(X)
X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

X = ZeroPadding2D(padding=(1, 1))(X)
X = Conv2D(8, kernel_size=(2,2), strides=(1,1))(X)
X = BatchNormalization(axis=3)(X)
X = Activation('relu')(X)
X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)

# FC
X  = Flatten()(X)
Y1 = Dense(6,activation=None)(X)
Y  = Activation("softmax")(Y1)

model = Model(inputs = X_input, outputs = Y, name='SIGNModel')
model.compile(optimizer = "sgd", loss = "categorical_crossentropy", metrics = ["accuracy"])
model.fit(X_train,Y_train, epochs = 10, batch_size=16 )
```

```
Epoch 1/10
1080/1080 [==============================] - 503s 466ms/step - loss: 1.7599 - acc: 0.2944
Epoch 2/10
1080/1080 [==============================] - 564s 522ms/step - loss: 1.1296 - acc: 0.5833
Epoch 3/10
1080/1080 [==============================] - 543s 503ms/step - loss: 0.7457 - acc: 0.7528
Epoch 4/10
1080/1080 [==============================] - 525s 486ms/step - loss: 0.5359 - acc: 0.8333
Epoch 5/10
1080/1080 [==============================] - 525s 487ms/step - loss: 0.4295 - acc: 0.8620
Epoch 6/10
1080/1080 [==============================] - 533s 493ms/step - loss: 0.3244 - acc: 0.9065
Epoch 7/10
1080/1080 [==============================] - 512s 474ms/step - loss: 0.2792 - acc: 0.9185
Epoch 8/10
1080/1080 [==============================] - 580s 537ms/step - loss: 0.2199 - acc: 0.9509
Epoch 9/10
1080/1080 [==============================] - 480s 444ms/step - loss: 0.1825 - acc: 0.9556
Epoch 10/10
1080/1080 [==============================] - 530s 490ms/step - loss: 0.1507 - acc: 0.9713
```

很明顯看到增加層數，抓取更多特徵，出現 overfitting 的問題，測試資料正確率大幅下降，因此在疊網路有時候修改參數會比新建網路更為重要，且需要耗費的時間為原本的 10 倍，效率太差，不適合筆電。

```
120/120 [==============================] - 1s 11ms/step
model
Loss = 0.5322820583979289
Test Accuracy = 0.8083333333333333
The figure is 2
```



ResNet50 的架構是由很多層 identity block 和 convolution block 堆疊而成

```
def ResNet50(input_shape = (64, 64, 3), classes = 6):
    """
    Implementation of the popular ResNet50 the following architecture:
    CONV2D -> BATCHNORM -> RELU -> MAXPOOL -> CONVBLOCK -> IDBLOCK*2 -> CONVBLOCK -> IDBLOCK*3
    -> CONVBLOCK -> IDBLOCK*5 -> CONVBLOCK -> IDBLOCK*2 -> AVGPOOL -> TOPLAYER

    Arguments:
    input_shape -- shape of the images of the dataset
    classes -- integer, number of classes

    Returns:
    model -- a Model() instance in Keras
    """

    # Define the input as a tensor with shape input_shape
    X_input = Input(input_shape)


    # Zero-Padding
    X = ZeroPadding2D((3, 3))(X_input)

    # Stage 1
    X = Conv2D(64, (7, 7), strides = (2, 2), name = 'conv1', kernel_initializer = glorot_uniform(seed=0))(X)
    X = BatchNormalization(axis = 3, name = 'bn_conv1')(X)
    X = Activation('relu')(X)
    X = MaxPooling2D((3, 3), strides=(2, 2))(X)

    # Stage 2
    X = convolutional_block(X, f = 3, filters = [64, 64, 256], stage = 2, block='a', s = 1)
    X = identity_block(X, 3, [64, 64, 256], stage=2, block='b')
    X = identity_block(X, 3, [64, 64, 256], stage=2, block='c')

    ### START CODE HERE ###

    # Stage 3 (≈4 lines)
    # The convolutional block uses three set of filters of size [128,128,512], "f" is 3, "s" is 2 and the block is "a".
    # The 3 identity blocks use three set of filters of size [128,128,512], "f" is 3 and the blocks are "b", "c" and "d"
    X = convolutional_block(X, f = 3, filters = [128, 128, 512], stage = 3, block='a', s = 2)
    X = identity_block(X, 3, [128, 128, 512], stage=3, block='b')
    X = identity_block(X, 3, [128, 128, 512], stage=3, block='c')
    X = identity_block(X, 3, [128, 128, 512], stage=3, block='d')

    # Stage 4 (≈6 lines)
    # The convolutional block uses three set of filters of size [256, 256, 1024], "f" is 3, "s" is 2 and the block is "a".
    # The 5 identity blocks use three set of filters of size [256, 256, 1024], "f" is 3 and the blocks are "b", "c", "d", "e" and "f".
    X = convolutional_block(X, f = 3, filters = [256, 256, 1024], stage = 4, block='a', s = 2)
    X = identity_block(X, 3, [256, 256, 1024], stage=4, block='b')
    X = identity_block(X, 3, [256, 256, 1024], stage=4, block='c')
    X = identity_block(X, 3, [256, 256, 1024], stage=4, block='d')
    X = identity_block(X, 3, [256, 256, 1024], stage=4, block='e')
    X = identity_block(X, 3, [256, 256, 1024], stage=4, block='f')

    # Stage 5 (≈3 lines)
    # The convolutional block uses three set of filters of size [512, 512, 2048], "f" is 3, "s" is 2 and the block is "a".
    # The 2 identity blocks use three set of filters of size [256, 256, 2048], "f" is 3 and the blocks are "b" and "c".
    X = convolutional_block(X, f = 3, filters = [512, 512, 2048], stage = 5, block='a', s = 2)
    X = identity_block(X, 3, [256, 256, 2048], stage=5, block='b')
    X = identity_block(X, 3, [256, 256, 2048], stage=5, block='c')

    # AVGPOOL (≈1 line). Use "X = AveragePooling2D(...)(X)"
    # The 2D Average Pooling uses a window of shape (2,2) and its name is "avg_pool".
    X = AveragePooling2D(pool_size=(2,2))(X)

    ### END CODE HERE ###

    # output layer
    X = Flatten()(X)
    X = Dense(classes, activation='softmax', name='fc' + str(classes), kernel_initializer = glorot_uniform(seed=0))(X)


    # Create model
    model = Model(inputs = X_input, outputs = X, name='ResNet50')

    return model
```

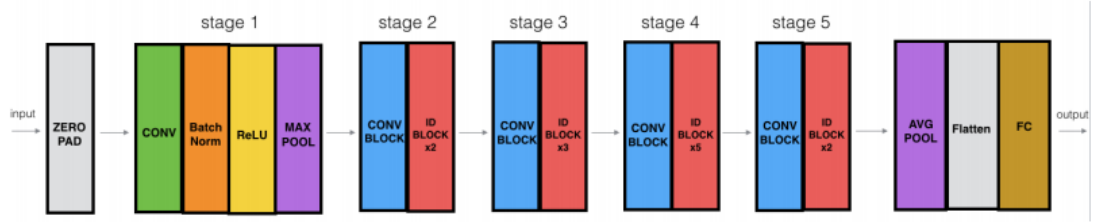


Figure 6: ResNet-50 model.

根據此圖建立起六個區域，使用 stage 區分區域。

```
def identity_block(X, f, filters, stage, block):
    """
    Implementation of the identity block as defined in Figure 4

    Arguments:
    X -- input tensor of shape (m, n_H_prev, n_W_prev, n_C_prev)
    f -- integer, specifying the shape of the middle CONV's window for the main path
    filters -- python list of integers, defining the number of filters in the CONV layers of the main path
    stage -- integer, used to name the layers, depending on their position in the network
    block -- string/character, used to name the layers, depending on their position in the network

    Returns:
    X -- output of the identity block, tensor of shape (n_H, n_W, n_C)
    """

    # defining name basis
    conv_name_base = 'res' + str(stage) + block + '_branch'
    bn_name_base = 'bn' + str(stage) + block + '_branch'

    # Retrieve Filters
    F1, F2, F3 = filters

    # Save the input value. You'll need this later to add back to the main path.
    X_shortcut = X

    # First component of main path
    X = (Conv2D(filters = F1, kernel_size = (1, 1), strides = (1,1), padding = 'valid',
               name = conv_name_base + '2a', kernel_initializer = glorot_uniform(seed=0))(X))
    X = BatchNormalization(axis = 3, name = bn_name_base + '2a')(X)
    X = Activation('relu')(X)

    ### START CODE HERE ###

    # Second component of main path (≈3 lines)
    X = (Conv2D(filters = F2, kernel_size = (f, f), strides = (1,1), padding = 'same',
               name = conv_name_base + '2b', kernel_initializer = glorot_uniform(seed=0))(X))
    X = BatchNormalization(axis = 3, name = bn_name_base + '2b')(X)
    X = Activation('relu')(X)

    # Third component of main path (≈2 lines)
    X = (Conv2D(filters = F3, kernel_size = (1, 1), strides = (1,1), padding = 'valid',
               name = conv_name_base + '2c', kernel_initializer = glorot_uniform(seed=0))(X))
    X = BatchNormalization(axis = 3, name = bn_name_base + '2c')(X)

    # Final step: Add shortcut value to main path, and pass it through a RELU activation (≈2 lines)
    X = Add()([X,X_shortcut])
    X = Activation('relu')(X)

    ### END CODE HERE ###

    return X
```

```
def convolutional_block(X, f, filters, stage, block, s = 2):
    """
    Implementation of the convolutional block as defined in Figure 4

    Arguments:
    X -- input tensor of shape (m, n_H_prev, n_W_prev, n_C_prev)
    f -- integer, specifying the shape of the middle CONV's window for the main path
    filters -- python list of integers, defining the number of filters in the CONV layers of the main path
    stage -- integer, used to name the layers, depending on their position in the network
    block -- string/character, used to name the layers, depending on their position in the network
    s -- Integer, specifying the stride to be used

    Returns:
    X -- output of the convolutional block, tensor of shape (n_H, n_W, n_C)
    """

    # defining name basis
    conv_name_base = 'res' + str(stage) + block + '_branch'
    bn_name_base = 'bn' + str(stage) + block + '_branch'

    # Retrieve Filters
    F1, F2, F3 = filters

    # Save the input value
    X_shortcut = X


    ##### MAIN PATH #####
    # First component of main path
    X = Conv2D(F1, (1, 1), strides = (s,s), name = conv_name_base + '2a', padding='valid', kernel_initializer = glorot
    X = BatchNormalization(axis = 3, name = bn_name_base + '2a')(X)
    X = Activation('relu')(X)

    ### START CODE HERE ###

    # Second component of main path (≈3 lines)
    X = (Conv2D(filters = F2, kernel_size = (f, f), strides = (1,1), padding = 'same',
               name = conv_name_base + '2b', kernel_initializer = glorot_uniform(seed=0))(X))
    X = BatchNormalization(axis = 3, name = bn_name_base + '2b')(X)
    X = Activation('relu')(X)

    # Third component of main path (≈2 lines)
    X = (Conv2D(filters = F3, kernel_size = (1, 1), strides = (1,1), padding = 'valid',
               name = conv_name_base + '2c', kernel_initializer = glorot_uniform(seed=0))(X))
    X = BatchNormalization(axis = 3, name = bn_name_base + '2c')(X)


    ##### SHORTCUT PATH #### (≈2 lines)
    X_shortcut =(Conv2D(filters = F3, kernel_size = (1, 1), strides = (s,s), padding = 'valid',
               name = conv_name_base + '1', kernel_initializer = glorot_uniform(seed=0))(X_shortcut))
    X_shortcut = BatchNormalization(axis = 3, name = bn_name_base + '1')(X_shortcut)

    # Final step: Add shortcut value to main path, and pass it through a RELU activation (≈2 lines)
    X = Add()([X,X_shortcut])
    X = Activation('relu')(X)

    ### END CODE HERE ###

    return X
```

```
In [22]: model.fit(X_train, Y_train, epochs = 2, batch_size = 32)

         Epoch 1/2
         1080/1080 [==============================] - 1678s 2s/step - loss: 3.1700 - acc: 0.2185
         Epoch 2/2
         1080/1080 [==============================] - 1649s 2s/step - loss: 2.3008 - acc: 0.3565

Out[22]: <keras.callbacks.History at 0x1a30194c6a0>

In [23]: preds = model.evaluate(X_test, Y_test)
         print ("Loss = " + str(preds[0]))
         print ("Test Accuracy = " + str(preds[1]))

         120/120 [==============================] - 6s 49ms/step
         Loss = 12.672466723124186
         Test Accuracy = 0.16666666716337203
```

可以看到使用 CPU 運算一次需要耗時近 1 小時

```
1 model.fit(X_train, Y_train, epochs = 20, batch_size = 32)

Epoch 1/20
1080/1080 [==============================] - 37s 34ms/step - loss: 1.0992 - accuracy: 0.6880
Epoch 2/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.9951 - accuracy: 0.6889
Epoch 3/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.9088 - accuracy: 0.7593
Epoch 4/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.4638 - accuracy: 0.8565
Epoch 5/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.4134 - accuracy: 0.8583
Epoch 6/20
1080/1080 [==============================] - 41s 38ms/step - loss: 0.2828 - accuracy: 0.9009
Epoch 7/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.1518 - accuracy: 0.9444
Epoch 8/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.1110 - accuracy: 0.9620
Epoch 9/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.0986 - accuracy: 0.9657
Epoch 10/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.0287 - accuracy: 0.9889
Epoch 11/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.0477 - accuracy: 0.9861
Epoch 12/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.1540 - accuracy: 0.9639
Epoch 13/20
1080/1080 [==============================] - 39s 36ms/step - loss: 0.1723 - accuracy: 0.9370
Epoch 14/20
1080/1080 [==============================] - 39s 36ms/step - loss: 0.2807 - accuracy: 0.9306
Epoch 15/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.6836 - accuracy: 0.7861
Epoch 16/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.4273 - accuracy: 0.8713
Epoch 17/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.1676 - accuracy: 0.9454
Epoch 18/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.1908 - accuracy: 0.9407
Epoch 19/20
1080/1080 [==============================] - 40s 37ms/step - loss: 0.0830 - accuracy: 0.9731
Epoch 20/20
1080/1080 [==============================] - 41s 38ms/step - loss: 0.0754 - accuracy: 0.9806

<keras.callbacks.callbacks.History at 0x1702b208108>

120/120 [==============================] - 1s 10ms/step
Loss = 0.3900323291619619
Test Accuracy = 0.8999999761581421
```

使用 GPU(GTX 1060)每個 epoch 僅需 40 秒，差距非常大。經過 20 個 epoch 測試資料正確率可以來到 90%左右。試著改變 mini batch：

```
1  model.fit(X_train, Y_train, epochs = 20, batch_size = 16)
```

```
Epoch 1/20
1080/1080 [==============================] - 59s 55ms/step - loss: 3.3638 - accuracy: 0.5463
Epoch 2/20
1080/1080 [==============================] - 51s 47ms/step - loss: 2.7547 - accuracy: 0.3713
Epoch 3/20
1080/1080 [==============================] - 54s 50ms/step - loss: 1.6521 - accuracy: 0.5787
Epoch 4/20
1080/1080 [==============================] - 57s 53ms/step - loss: 1.3483 - accuracy: 0.6454
Epoch 5/20
1080/1080 [==============================] - 55s 51ms/step - loss: 1.5333 - accuracy: 0.6796
Epoch 6/20
1080/1080 [==============================] - 54s 50ms/step - loss: 1.2316 - accuracy: 0.6889
Epoch 7/20
1080/1080 [==============================] - 55s 51ms/step - loss: 0.9045 - accuracy: 0.7500
Epoch 8/20
1080/1080 [==============================] - 52s 49ms/step - loss: 0.6902 - accuracy: 0.8139
Epoch 9/20
1080/1080 [==============================] - 56s 52ms/step - loss: 0.6927 - accuracy: 0.8463
Epoch 10/20
1080/1080 [==============================] - 56s 52ms/step - loss: 0.3933 - accuracy: 0.8741
Epoch 11/20
1080/1080 [==============================] - 54s 50ms/step - loss: 0.3813 - accuracy: 0.8852
Epoch 12/20
1080/1080 [==============================] - 55s 51ms/step - loss: 0.1762 - accuracy: 0.9435
Epoch 13/20
1080/1080 [==============================] - 56s 52ms/step - loss: 0.1165 - accuracy: 0.9583
Epoch 14/20
1080/1080 [==============================] - 60s 55ms/step - loss: 0.1654 - accuracy: 0.9556
Epoch 15/20
1080/1080 [==============================] - 57s 53ms/step - loss: 0.1670 - accuracy: 0.9352
Epoch 16/20
1080/1080 [==============================] - 55s 51ms/step - loss: 0.0760 - accuracy: 0.9713
Epoch 17/20
1080/1080 [==============================] - 55s 51ms/step - loss: 0.0777 - accuracy: 0.9722
Epoch 18/20
1080/1080 [==============================] - 56s 52ms/step - loss: 0.0653 - accuracy: 0.9778
Epoch 19/20
1080/1080 [==============================] - 55s 51ms/step - loss: 0.0401 - accuracy: 0.9880
Epoch 20/20
1080/1080 [==============================] - 56s 51ms/step - loss: 0.0920 - accuracy: 0.9657

<keras.callbacks.callbacks.History at 0x1703b3e99c8>
```

minibatch=16 每個 epoch 所耗的時間增加，但訓練資料正確率下降，Loss 也較大。由於 loss 還有下降空間，理論上增加 epoch 可以提高正確率。

將 minibatch 調整為 8

```
1 model.fit(X_train, Y_train, epochs = 20, batch_size = 8)
```

```
Epoch 1/20
1080/1080 [==============================] - 76s 70ms/step - loss: 0.9931 - accuracy: 0.7880
Epoch 2/20
1080/1080 [==============================] - 76s 71ms/step - loss: 0.7530 - accuracy: 0.8120
Epoch 3/20
1080/1080 [==============================] - 75s 69ms/step - loss: 0.4147 - accuracy: 0.8981
Epoch 4/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.6036 - accuracy: 0.8250
Epoch 5/20
1080/1080 [==============================] - 79s 73ms/step - loss: 0.4796 - accuracy: 0.8694
Epoch 6/20
1080/1080 [==============================] - 77s 71ms/step - loss: 0.2744 - accuracy: 0.9222
Epoch 7/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.2376 - accuracy: 0.9389
Epoch 8/20
1080/1080 [==============================] - 73s 68ms/step - loss: 0.1896 - accuracy: 0.9556
Epoch 9/20
1080/1080 [==============================] - 74s 68ms/step - loss: 0.2111 - accuracy: 0.9444
Epoch 10/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.1252 - accuracy: 0.9639
Epoch 11/20
1080/1080 [==============================] - 74s 68ms/step - loss: 0.2281 - accuracy: 0.9417
Epoch 12/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.2118 - accuracy: 0.9509
Epoch 13/20
1080/1080 [==============================] - 73s 67ms/step - loss: 0.0909 - accuracy: 0.9796
Epoch 14/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.0430 - accuracy: 0.9861
Epoch 15/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.1219 - accuracy: 0.9630
Epoch 16/20
1080/1080 [==============================] - 74s 68ms/step - loss: 0.0475 - accuracy: 0.9796
Epoch 17/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.0809 - accuracy: 0.9769
Epoch 18/20
1080/1080 [==============================] - 74s 69ms/step - loss: 0.1478 - accuracy: 0.9620
Epoch 19/20
1080/1080 [==============================] - 73s 67ms/step - loss: 0.1686 - accuracy: 0.9463
Epoch 20/20
1080/1080 [==============================] - 72s 67ms/step - loss: 0.2853 - accuracy: 0.9528

<keras.callbacks.callbacks.History at 0x17039a772c8>
```

loss 下降緩慢，也因此訓練資料正確率是最低的，需要更多的 epoch 才有辦法提高正確率，但耗時也需要更久，考量到效率問題，minibatch 設定為 32 是很好的參數，既能省下時間，正確率表現也不俗